

Contents

Sequencing, Segment Breakage, and Replanning	1
0. Vocabulary (read this first)	1
1. Lifecycle: from master segment to executed job	2
2. Boustrophedon sequencer — grouping (always runs)	2
3. Segment breakage (invalidation and split)	3
4. Transit cost metric (used for ordering, not for driving)	4
5. ATSP sequencing (default job order)	5
6. How jobs connect in space (execution)	6
7. When the schedule is rebuilt (“replan”)	7
8. Reachability filter (after ATSP)	8
9. Retries and limits (exact counts)	8
10. Side-by-side: two sequencing modes	8
11. Exam-style answers	9
12. Code index	10

Sequencing, Segment Breakage, and Replanning

How work is grouped, ordered, connected, and retried. Defaults: `gpr_bringup/config/gpr_coverage.yaml`.

Default: `sequencer.use_boustrophedon: true` — passes are merged, then ordered lane-by-lane (greedy). Set `false` to order jobs with **ATSP** (OR-Tools, 2 s budget).

Related: `ALGORITHM.md` · PDF build: `README.md`

0. Vocabulary (read this first)

The code uses two different ideas that are easy to conflate:

Term in conversation	What it actually is in code
Boustrophedon planner	Builds the <i>initial</i> zig-zag lanes and 1 m master segments (mission start, obstacle-free).
Boustrophedon sequencer	Takes <i>schedulable catalog segments</i> and merges adjacent pieces on the same lane into passes . Optionally orders those passes (only in one mode).
ATSP solver	Orders jobs (passes + orphan segments) by approximate transit cost.
A* planner	Plans the actual collision-free polyline the robot drives between jobs (once per job).

So: **boustrophedon sequencer** the whole mission order. It always **groups** passes; `use_boustrophedon` chooses greedy lane order vs ATSP order.

1. Lifecycle: from master segment to executed job

Mission start

```
BoustrophedonPlanner.generate_segments()
    → N master segments (Baseline, 1 m pieces, lane_index set)
```

Robot moves, LiDAR updates grid

```
SegmentCatalog.update_blocked(grid)
    → split / block / trim centerlines (§3)
```

Each BT cycle (2 Hz)

```
schedulable_segments() → open, unblocked Pending only
if schedule empty or stale → recompute_schedule()
    BoustrophedonSequencer.plan() → CoveragePass list (§2)
    ATSP or boustrophedon order → queue of CoverageJob (§4-5)
pop_next_job() → execute one job
    A* transit to entry (if far) → §6
    FollowPath on coverage prefix → §7
```

One job at a time. The queue front is executed; there is no parallel coverage.

2. Boustrophedon sequencer — grouping (always runs)

File: gpr_planning/src/boustrophedon_sequencer.cpp

Called from: PlanningEngine::recompute_schedule() on every full reschedule.

2.1 Input

schedulable_segments() from the catalog:

- outcome == Pending
- blocked == false
- centerline.size() >= 2
- **PartiallyCompleted is NOT schedulable** (tail not re-queued in current build)
- OARP ranks and split children qualify if pending and free

Segments without lane_index skip pass merging (they enter ATSP as orphans).

2.2 Within one lane

1. **Drive direction** for the lane (fixed for pass geometry):

- lane_index % 2 == 0 → Forward along centerline
- lane_index % 2 == 1 → Reverse

2. **Sort** segments along the lane:

- scan_direction == "x": sort by front pose **y** (after orienting to lane direction)
- scan_direction == "y": sort by front pose **x**

3. **Merge into runs:** consecutive sorted segments join one run if any endpoint of the previous piece is within `merge_gap_m` (0.2 m) of any endpoint of the next:

$\text{min distance among \{back/front of A\} \times \{back/front of B\} \leq 0.2 \text{ m}}$

4. **Flush run $\rightarrow$ CoveragePass:**

- `centerline` = merged polyline along lane direction
- `segment_ids` = all constituent catalog ids
- `job.segment_id` = `make_pass_job_id(lane_index, pass_ordinal)` (synthetic PASS id)
- `job.covers` = `segment_ids`
- `job.direction` = lane direction from step 1

After a split: two free fragments on the same lane that are **not** within 0.2 m of each other become **two passes** (two jobs), not one.

2.3 What the sequencer does NOT do (default yaml)

With `use_boustrophedon_sequencer: false`, `recompute_schedule` calls:

```
boustrophedon_sequencer_.plan(schedulable, config, std::nullopt);
```

`robot_pose = nullopt` $\rightarrow$ passes are **not** ordered by greedy nearest-neighbor. Only grouping happens; order is left to ATSP.

2.4 Boustrophedon ordering mode (`use_boustrophedon: true`)

Same grouping, then **greedy nearest pass** ordering:

- Cursor starts at robot pose.
- Each step: among remaining passes, pick direction with `prefer_drive_direction` (entry closer to cursor).
- **Cost** to rank passes:
 - If grid + astar available: `astar.transit_cost(cursor, entry)` (§4.1)
 - Else: Euclidean `hypot(entry - cursor)`
- Append chosen pass to schedule; cursor = pass exit pose.
- Repeat until no passes left.
- Any schedulable segment **not** in any pass $\rightarrow$ **ATSP fallback** on that subset only.

This mode is **not** the default in `gpr_coverage.yaml`.

3. Segment breakage (invalidation and split)

Files: `path_invalidator.cpp`, `segment_catalog.cpp`

3.1 When splits run

Every `UpdateSegmentCatalog` BT tick applies `catalog.update_blocked(latest_grid)` to all **Pending** segments.

Completed / Skipped / PartiallyCompleted are not re-split.

3.2 Collision test

Each centerline is walked on the **planning grid** (/local_occupancy_grid_planning):

- Sample along edges every $\max(0.05 \text{ m}, 0.5 \times \text{resolution})$.
- At each sample, disk collision with radius **footprint_radius** (0.22 m).
- Cell blocks if cost ≥ 50 (and not ignored at boundary).

3.3 Split outcomes

Grid vs centerline	Catalog result
Fully free (one part, full length)	Trim centerline to free polyline; may set blocked if still colliding
Fully blocked (no free part)	blocked = true , segment kept
Partially blocked (multiple free runs)	Remove parent; create Split children

Children:

- **id** = **make_split_segment_id**(baseline_id, ordinal) — new id
- **baseline_id** = parent's effective baseline (master id preserved for metrics/viz)
- **lane_index** inherited
- Fragments shorter than **min_split_length_m** (0.25 m) **discarded**
- If every fragment too short $\rightarrow$ parent marked **blocked**

Frozen segments: if attempted && **blocked**, no further split (recovery already tried).

3.4 Connectivity after breakage

- **Along-lane connectivity** is rediscovered at reschedule: if two split children endpoints are within 0.2 m, they merge back into **one pass**.
- **Across lanes** there is **no** fixed connector segment. The robot uses **A* transit** from exit of job J to entry of job J+1.
- Initial plan connectors (**BoustrophedonPlanner.generate()**) are for **/initial_coverage_path** visualization only.

3.5 Identity chain

```
Master baseline (1 m, id = make_segment_id(lane, s))
  → split child (id = SPLIT..., baseline_id = master id)
  → pass job (id = PASS..., covers = {master and/or split ids})
```

job_still_actionable matches jobs to catalog via **segment_id**, **covers**, **baseline_id**, and split children.

4. Transit cost metric (used for ordering, not for driving)

File: **astar_grid_planner.cpp** $\rightarrow$ **transit_cost()**

This is **not** A* search. It is a cheap estimate because ATSP builds an $N \times N$ matrix.

`d = Euclidean distance from_pose to to_pose`

`blocked = straight segment from-to collides on planning grid
(same footprint + threshold as invalidator)`

`cost = d * 4.0 if blocked
 d otherwise`

`4.0 = astar.blocked_transit_penalty.`

Important: the robot does **not** follow this straight line. Execution uses `plan_path()` (8-connected A*) for the real transit polyline.

5. ATSP sequencing (default job order)

Files: `or_tools_atsp_solver.cpp`, `heuristic_atsp_solver.cpp`, `sequence_solver_utils.cpp`

Solver selection: OR-Tools if compiled (GPR_HAS_ORTOOLS); else heuristic only.

5.1 ATSP input set (`use_boustrophedon: false`)

`atsp_segments = [pass_to_schedulable_segment(pass) for each pass]
 + [orphan schedulable segments not in any pass.segment_ids]`

Each **pass** becomes one ATSP node (centerline = merged pass line, id = PASS job id).

Orphan **1 m** (or split) segments are separate ATSP nodes.

5.2 Directed jobs (two orientations per segment)

For each ATSP node segment, unless direction is fixed by pass metadata:

- **Forward job:** entry = front pose, exit = back pose
- **Reverse job:** entry = back pose, exit = front pose

Pass nodes have **fixed direction** from pass merge (`attach_pass_metadata / fixed_directions`).

5.3 Cost matrix

Nodes:

- 0 = depot (robot pose)
- 1..N = directed jobs
- optional N+1 = home depot if return-home pose recorded

Arc weight from node i to j:

`matrix[i][j] = round(1000 * transit_cost(exit_pose(i), entry_pose(j)))`

Non-finite $\rightarrow 10^9$.

5.4 OR-Tools model

- Single vehicle routing (ATSP with fixed start).

- **Disjunction** per physical `segment_id`: if both Forward and Reverse nodes exist, at most one is visited; skipping costs 10^8 .
- Strategy: `PATH_CHEAPEST_ARC`.
- Time limit: `sequencer.time_limit_sec` (2.0 s).
- **One solve attempt**. If `SolveWithParameters` returns null $\rightarrow$ **HeuristicAtspSolver** (no second OR-Tools retry).

5.5 Heuristic fallback

Greedy nearest-neighbor on **remaining segment ids** (not passes):

```
while segments remain:
    pick directed job minimizing transit_cost(current, entry)
    if last job and return-home enabled:
        pick job minimizing cost(current→entry) + cost(exit→home)
    append job; current = exit
```

May return a **partial** schedule if no finite-cost candidate exists.

5.6 Output queue

Ordered vector<CoverageJob>. `attach_pass_metadata` fills `job.covers` for PASS ids.

Next job = `pop_next_job()` takes **front** of queue after reachability filter (§8).

6. How jobs connect in space (execution)

6.1 Entry and exit poses

For segment S and direction D:

- Forward: entry = front, exit = back
- Reverse: entry = back, exit = front

For pass job: entry/exit from **merged** pass centerline and pass `job.direction`.

6.2 Transit leg

If `distance(robot, entry) > transit_skip_distance` (0.35 m):

1. `transit_to_current_job = A* plan_path(robot, entry)` on planning grid
2. Nav2 FollowPath on A* polyline
3. Trace tagged `ExecutionTracePhase::Transit` (not counted for swath coverage)

If A* fails or transit polyline not followable $\rightarrow$ job **FAILURE** $\rightarrow$ recovery (§9).

If `distance < 0.35 m`: skip transit; start coverage immediately.

6.3 Coverage leg

1. `trim_polyline_ahead(job_polyline, robot, prune=0.15 m)`
2. `free_subpolylines` on planning grid $\rightarrow$ take **longest** free prefix

3. Nav2 FollowPath on that prefix
4. Trace tagged `ExecutionTracePhase::Coverage`

6.4 Between jobs

There is no stored “edge” in the catalog. Connection is implicit:

```
exit_pose(job_k) --[A* transit]--> entry_pose(job_{k+1})
```

Schedule order defines $k \rightarrow k+1$. Replanning may change the whole queue when the map changes.

7. When the schedule is rebuilt (“replan”)

Not continuous A* improvement until goal. **Discrete** full reschedule events:

7.1 `schedule_needs_rebuild()` true when

- Current job no longer actionable (all covers blocked / gone), or
- Some queued job no longer actionable, or
- Queue empty **and** schedulable segments exist, or
- Queue empty **and** OARP may still inject work (catalog revision not yet tried idle)

7.2 `RecomputeSchedule` BT node

Runs async via `PlanningWorker` unless:

- `has_pending_jobs()` and not `force_replan` → skip (keep current queue)
- `reachability_exhausted()` → skip
- `!schedule_needs_rebuild()` and not forced → skip

On success: new `schedule_`, `scheduled_passes_`, `schedule_revision_++`.

7.3 Interrupt `replan` (during coverage only)

If map update invalidates **active coverage job** while Nav2 is driving:

- Cancel control
- Release job if dead
- **Urgent** `RecomputeSchedule` (`force_replan=true`)
- **Not** triggered during committed transit leg

7.4 Held job during rebuild

If robot is mid-job, `recompute_schedule` keeps `current_job_` and removes overlapping jobs from the new queue.

8. Reachability filter (after ATSP)

If `require_reachable_transit: true` (default):

Each job in queue must pass `is_job_reachable(robot)`:

```
if dist(robot, entry) < 0.35 m:
    reachable iff coverage prefix followable on grid
else:
    reachable iff A* path exists AND transit path followable
```

Unreachable jobs are **dropped** from queue (not deferred to back).

If queue empty but schedulable segments remain:

- `block_unreachable_schedulable` marks segment **blocked** when **neither** forward nor reverse is reachable
- `reachability_exhausted` may end mission loop

9. Retries and limits (exact counts)

Mechanism	Limit	What happens
Job execution failure (Nav2 abort, A* fail, coverage < 35%)	12 consecutive (<code>kMaxRecoverySkips</code>)	<code>SkipCurrentJob</code> → mark blocked → <code>force_replan</code> ; mission ends if count 12
Successful job	—	<code>recovery_skip_count</code> reset to 0
OR-Tools solve	1 attempt per reschedule	Falls back to heuristic tour (not a retry of OR-Tools)
OARP rank injection	max_replan_generations = 2 per mission	After 2 batches, no new OARP ranks
OARP batch acceptance	1 A* check	If robot cannot reach first rank entry, entire batch rejected
Idle empty schedule + OARP possible	1 replan per catalog revision	<code>last_empty_schedule_catalog_rev_</code> prevents spin
pop_next_job unreachable	No cap	Drops job from front, sets <code>catalog_changed</code> , tries next
PartiallyCompleted tail	0 re-queue	Not schedulable until policy changes

There is **no** “retry this job 3 times” counter per segment. Failure → blocked + reschedule other work.

10. Side-by-side: two sequencing modes

<code>use_boustrophedon: false (ATSP)</code>	<code>use_boustrophedon: true (default)</code>
Pass Yes	Yes
merg-	
ing	
Pass ATSP on passes + orphans	Greedy nearest pass order
or-	
der	
Orphan ATSP	ATSP fallback
not	
in	
a	
pass	
Cost <code>transit_cost</code> matrix + OR-Tools	Greedy uses same <code>transit_cost</code>
met-	
ric	
Direction Fixed at merge (even/odd lane)	Re-chosen per step
per	(<code>prefer_drive_direction</code>)
pass	
Return ATSP extra depot node	Greedy only (ATSP on orphans)
home	
in	
or-	
der	

Both modes use the same:

- Segment splitting rules
- Segment catalog updates (`update_blocked`, swath coverage, schedulability)
- A* for actual transit
- Swath coverage completion (85% / 35%)
- Recovery and retry limits

Segment **colors and queue membership** do not depend on which sequencer ordered the jobs.

11. Exam-style answers

Q: How is the next segment chosen?

A: The next **job** is the front of the schedule queue after ATSP (or boustrophedon greedy) and reachability filtering—not a per-tick re-solve.

Q: What metric orders jobs?

A: Sum of `transit_cost` along the tour (OR-Tools minimizes). Per arc: Euclidean meters, $\times 4$ if straight line crosses occupied grid cells.

Q: Is A* used in ordering?

A: No. Only in execution and reachability checks.

Q: What happens when A* fails?

A: Job fails → blocked → schedule recomputed. After 12 consecutive failures, mission stops.

Q: How do broken segments reconnect?

A: Colinear free fragments on the same lane may merge into one pass (0.2 m gap). Non-adjacent fragments stay separate jobs; robot A* transits between them.

Q: Boustrophedon vs ATSP?

A: Boustrophedon **sequencer** groups lane pieces into passes. With `use_boustrophedon: true` (default), passes are ordered greedily by lane; with `false`, **ATSP** orders passes and orphans. Grouping is unchanged.

12. Code index

Concern	File
Pass merge	<code>boustrophedon_sequencer.cpp</code>
Reschedule orchestration	<code>planning_engine.cpp</code> → <code>recompute_schedule</code>
ATSP	<code>or_tools_atsp_solver.cpp</code> , <code>heuristic_atsp_solver.cpp</code>
Transit cost	<code>astar_grid_planner.cpp</code> → <code>transit_cost</code> , <code>plan_path</code>
Split	<code>segment_catalog.cpp</code> → <code>apply_split_update</code>
Execute + recovery	<code>mission_behaviors.cpp</code> → <code>ExecuteNextJob</code> , <code>SkipCurrentJob</code>
Retry cap	<code>mission_context.hpp</code> → <code>kMaxRecoverySkips = 12</code>